

# Public Key Crypto notes - 06

By contributors

March 16. 2026

## 1 Proof of knowledge protocols

- Goal: Design a signature scheme that utilises the discrete log problem.
- Idea: Digital signature: Proof of knowledge of secret key without telling any information about it.
- Example: Ali Baba's cave: Bob can only meet Alice from  $A$  when entering from  $B$  if he knows the password. Alice does not hear the password.
- The probability of passing is 50%. If Bob passes the test two times, 25%, etc.
- Given  $n$  tests, Bob can pass the test by randomly guessing with  $\frac{1}{2^n}$  probability.
- This is a proof to the validator, Alice. For outside observers, no one can tell whether Bob really knows the password.
- Three steps:
  1. Commitment: When the prover makes an action (Bob enters the cave).
  2. Challenge: When the validator asks the direction from Bob ("*Enter from A and go through B*").
  3. Response: When Bob comes from the corresponding direction.

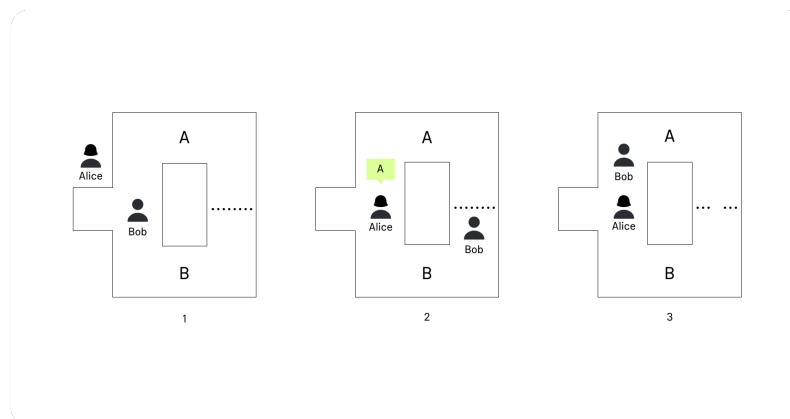


Figure 1: Ali Baba cave example

## 1.1 Schnorr identification scheme

Let  $pk = (p, g, q, h = g^x), sk = x$ .

- Goal: Prove the knowledge of  $sk$  under  $pk$  without revealing any information on  $sk$ .
- Protocol:
  1. Prover chooses  $k \in_R \mathbb{Z}_q$  and  $I = g^k$  and sends  $I$  to the verifier. (Commitment)
  2. Verifier chooses  $r \in_R \mathbb{Z}_q$  and sends this to the Prover. (Challenge)
  3. Prover computes  $s = rx + k$  and sends  $s$  to the Verifier. (Response)
  4. Verifier checks:  $I = g^s \cdot h^{-r}$ . If yes, the proof is accepted.
- Correctness:  $g^s \cdot h^{-r} = g^{rx+k} \cdot (g^x)^{-r} = g^{rx+k-rx} = g^k = I$

**Theorem 1.1.** If the Prover can give a valid response for any challenge, then the Prover knows  $x$ .

*Proof.* Let  $I = g^k$  and let  $r_1, r_2$  be two *different* challenges with respect to  $I$  and let  $s_1, s_2$  be the two responses.

$$s_1 = r_1x + k \tag{1}$$

$$s_2 = r_2x + k \tag{2}$$

$$s_1 - s_2 = (r_1 - r_2)x \tag{3}$$

So:

$$x = \frac{s_1 - s_2}{r_1 - r_2}$$

□

**Remark 1.1.** If the Prover knows  $r$  in advance (before the commitment), they can generate a valid response  $s$  without knowing secret  $x$ .

Indeed: For a given challenge  $r$ , let  $s \in_R \mathbb{Z}_q$  and  $I = g^s \cdot g^{-r}$ .

Note that this is an interactive protocol, since the Verifier challenges the Prover. The Verifier's role is to generate the challenge. How can we make it non-interactive?

## 1.2 Fiat-Shamir transform

Replace the challenge  $r$  with  $r = H(I)$ , where  $H$  is a secure cryptographic hash function.

To prove security, we use the Random Oracle Model (ROM), where  $H(\cdot)$  is replaced by a random oracle.

**Definition 1.1** (Random Oracle Model (ROM)). For a given input  $x$ , if it has not yet been queried, then let  $r \in_R \mathbb{Z}_q$ , and put  $r = \mathcal{O}(x)$ , where  $\mathcal{O}$  is the random oracle. If it has been queried, return the same output.

### 1.3 Schnorr signature scheme

- Gen:
  - Input:  $1^n$
  - Output:  $pk = (p, g, q, h = g^x), sk = x$
- Sig:
  - Input:  $sk, m$
  - Output:  $(I, s)$ , where:

$$k \in_R \mathbb{Z}_q, \tag{4}$$

$$I = g^k, \tag{5}$$

$$r = H(I, m), \tag{6}$$

$$s = rx + k \tag{7}$$

- Ver:
  - Input:  $pk, m, (I, s)$
  - Output: Compute  $r = H(I, m)$  and check whether  $I = g^s \cdot h^{-r}$

**Theorem 1.2.** The Schnorr signature scheme is secure, provided that  $H(\cdot)$  is modelled as a random oracle and the discrete logarithm problem is computationally hard.

*Proof.* The proof uses the rewind technique. We prove by reduction. Assume we have a PPT adversary  $A$  who can generate valid signatures with non-negligible probability. We use this  $A$  to compute the discrete logarithm.

For simplicity, we assume that  $A$  is *perfect*, meaning that it *always* returns a valid signature. Reduction: Given  $pk = (p, g, q, h = g^x)$ . Our goal is to learn  $x$ . TODO: Ábra a fűzetben

1. Give  $pk$  to  $A$ .
2. Give  $A$  oracle access to  $\mathcal{O}$ , and let  $Q$  be the set of queries submitted by  $A$ .
3.  $A$  generates  $I$  and  $m$  with  $(I, m) \notin R$  and queries the output  $r = \mathcal{O}(I, m)$ .
4.  $A$  returns  $(I, s)$ .

Now, rewind it to step 3. and get  $(i, s', r')$  at step 4. We have valid  $(r, s)$  and  $(r', s')$ , with the same  $I$ , i.e.  $s = rx + k$  and  $s' = r'x + k$  with some  $k$ , such that  $I = g^k$ .

This means that we can compute  $x$ . □

**Remark 1.2.** This signature algorithm has been patented (expired in 2010). Instead, NIST designed a new and free signature algorithm called Digital Signature Algorithm (DSA).